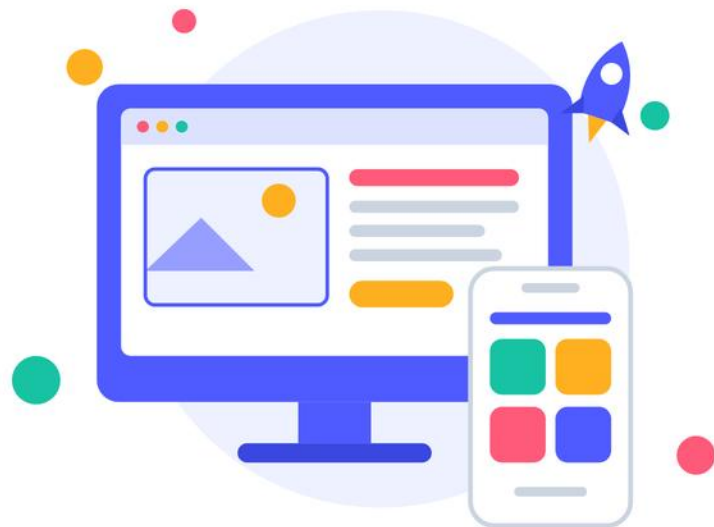


WEEK 01 · HTML I · THE DOCUMENT · GIT

WHAT THE BROWSER ACTUALLY DOES

You already program. Today you type your first page from an empty file, find out what the browser does with it, and put it somewhere you can never lose it.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Every great developer you know got there by solving problems they were unqualified to solve until they actually did it.

PATRICK MCKENZIE

WHERE WE ARE

Week 1 of 12. Nothing to build on yet — this is the ground floor.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: an empty file · div soup · three broken paths · one question to Claude · init to SHA. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

Your repo goes live today and every week lands in it. Today's page is next week's starting point.

MINUTE 2 · THE PROBLEM

YOU PRESS ENTER. THEN WHAT?

Between the keypress and the first pixel, six separate things happen — on at least three different machines. Six words, and you will meet the request, the response and the status again in week 11, in the Network tab.

You have done this ten thousand times and never once been told what it does. That is not a gap in your knowledge — it is a gap in how programming is taught.

Name them. You have thirty seconds and a neighbour.

THE WHOLE TRIP, ONCE

1 Request

a few lines of plain text: `GET /courses HTTP/1.1`

2 DNS

the host name becomes an address — a connection opens to a number, not a name

3 TLS

the connection is encrypted; the S in https

4 Response

a status, headers, and a body that is also text — `200` fine, `404` no such thing

5 Parse

the text becomes a tree: the DOM, the tree you walked in data structures

6 Paint

layout decides where every box goes, paint makes pixels

Six words: request, DNS, TLS, response, parse, paint.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment — no CSS, and it is not supposed to have any

↶ reload

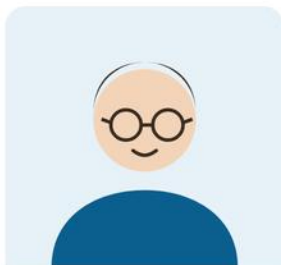
open ↷

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me



Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me



I came to programming through a statistics course I did not enjoy and a class we I did. I like problems where the answer is checkable, which is why I have some trash blog designs — and why this website is going to be unrepresentative.

I am writing this page in the first act of the course. Even so, enjoy this, and it is the first commit of the thing I will be examined on.

One thing that surprised me today

HTML, never expects an error. It expects what you wrote and renders something else. That is why we can't validate.

targets/week-01.png · 1280

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

- [About me](#)
- [What I am learning](#)
- [Contact me](#)

375 · nothing overflows

► **THE POINT** — this page looks like 1996 and validates clean, and a screen reader, Google and week 9's JavaScript can all find their way around it. You type every piece of it today: once in a cycle, once in the assignment. Then you push it, and it is the first commit of the thing you will be examined on.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1-5 · PRIVATE TUTOR

ALLOWED

Ask it what `section` means, why a `nav` wants a list, what the validator's message says. Ask it to criticise markup *you* typed. Then check the answer in the validator, in DevTools, or on MDN — not in your gut.

NOT ALLOWED

Pasting generated markup into the assignment. **You type every character.** What to check this week: every claim, against the primary source (MDN, the validator) — especially the one it sounds surest about.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one question, twice, in whatever chat you have.

CYCLE 1 · 25 MIN

FROM AN EMPTY FILE TO A SKELETON

A white rectangle in Live Server. Ten lines later it is a page with a title in the tab, a heading, two paragraphs, a list and a footer — and you will have watched one missing character rewrite the whole tree.

1 I DO

10 MIN

3 on the next two slides, 7 in the editor · live-code/01-empty-file-to-skeleton · doctype, lang, charset, viewport, title · main, h1, p, ul, footer

2 WE DO

6 MIN

you type it in your practice folder, with your name · compare with checkpoint 1

3 YOU DO

7 MIN

lab/warmup/ — make the file look like target.png · one element you have to guess

4 DEBRIEF

2 MIN

the unclosed tag · what the head shows · ↗ reference

YOU DO · AI-FREE

CYCLE 1 · BEFORE YOU TYPE

THE SMALLEST PAGE THAT ACTUALLY WORKS

Eight lines. Five of them exist for reasons you cannot see by looking at the result — and every one of those five breaks something specific when it is missing.

Which five? And what does each one break? Hold your answer — you will type every one of them in the next seven minutes, and watch one of them break.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Maya Cohen</title>
  </head>
  <body></body>
</html>
```

CYCLE 1 · BEFORE YOU TYPE

ELEMENTS NEST. THEY NEVER OVERLAP.

CROSSED

```
<p><strong>Careful</p></strong>
```

NESTED

```
<p><strong>Careful</strong></p>
```

Close elements in the reverse order you opened them (LIFO).

You know LIFO. It is the same stack you push and pop. The browser keeps one while it parses, and it will silently repair yours — into something you did not write.

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

Noa Levi

Third-year information systems student. Fluent in C# and SQL, new to the browser.

This page was typed from an empty file, one line at a time, in the first hour of the course.

- What I already write: C#, SQL, a little Python.
- What I am learning now: what the browser does with the text I send it.
- What I want by week 12: one application I can explain line by line.

Frontend Development 2027, Ruppin Academic Center. Typed in class.

Compare — `live-code/01-empty-file-to-skeleton/checkpoint.png` is this picture at 1280; `live-code/01-empty-file-to-skeleton/end/` is the code behind it. Your name in the tab, the paragraph the same size as the list, and zero errors from `npx html-validate@9`.

CYCLE 2 · 25 MIN

FROM DIV SOUP TO LANDMARKS

A page that is all `div`. It validates, it looks fine, and a screen reader finds nothing in it. Same pixels, different tree — and at the end, three mistakes, seven validator lines, and why fixing the first one removes five.

1 I DO

10 MIN

3 on the next three slides, 7 in the editor · live-code/02-div-soup-to-landmarks · header, nav > ul > li > a, main, section + h2, aside, footer · strong, not b · the validator on a wrong state

2 WE DO

5 MIN

you type it on your copy · compare with checkpoint 2 · `$$('header, nav, main, footer').length` is 4

3 YOU DO

8 MIN

your own page: header, nav list, two sections with h2, an aside · AI as tutor, checked in the accessibility tree

4 DEBRIEF

2 MIN

`div class="header"` · one main · ↗ reference

YOU DO · AI AS TUTOR

CYCLE 2 · BEFORE YOU TYPE

TWO PAGES. IDENTICAL PIXELS.

Press "show landmarks"

↺ reload

open ↗

Ruppín Coffee Club

[Opening hours](#) [This week's beans](#)

Opening hours

Sunday to Thursday, 08:00–16:00. Closed on exam days, obviously.

This week's beans

Ethiopia, Guji. Tastes of stone fruit and of someone else paying.

Trivia: the first webcam watched a coffee pot in Cambridge.

Built by second-years who should have been revising.

show landmarks

Press the button.

► **HANDS-ON** `demos/03-semantics/` — Press `show landmarks` here, then open `02-semantic/` in a second tab and press it there.

THE FOUR YOU WILL USE ON EVERY PAGE

- <header>** Introductory content for the page or for a section. Usually the title and the nav.
- <nav>** A block of *major* navigation links.
- <main>** The one region unique to this page. **Exactly one per document.**
- <footer>** Closing content: author, copyright, related links.

Not every link list is a nav.



A NAV IS A LIST OF LINKS, AND LISTS NEST

THE ONE THE ASSIGNMENT ASKS FOR

```
1 <nav aria-label="Main">
2   <ul>
3     <li><a href="#about">About me</a></li>
4     <li><a href="#work">Projects</a></li>
5     <li><a href="contact.html">Contact</a></li>
6   </ul>
7 </nav>
```

`nav` > `ul` > `li` > `a`. Every link is a list item; nothing else goes between them.

ORDERED, AND NESTED

```
1 <ol>
2   <li>Install VS Code
3     <ul>
4       <li>macOS: drag to Applications</li>
5       <li>Windows: run the installer</li>
6     </ul>
7   </li>
8   <li>Install Node</li>
9 </ol>
10
11 <ol start="7"><li>...continues at 7</li></ol>
```

A nested list goes **inside an `li`**, never between two of them. `ol` when the order is a fact; `ul` when it is not.

SECTION, ARTICLE, ASIDE

- `<section>`** A thematic group of content. It should have a heading.
- `<article>`** Something that stands on its own: a post, a card, a review, a comment.
- `<aside>`** Tangentially related — a sidebar, a pull quote, a "did you know".
- `<div>`** No meaning at all. A hook for styling. Still legitimate — just honest.

This content should be meaningful on its own, even if separated from the rest of the webpage.

 LOOKS BOLD. IS IMPORTANT.

PRESENTATION

```
<p>Deadline: <b>Sunday</b></p>  
<p><i>Do not be late</i></p>
```

MEANING

```
<p>Deadline: <strong>Sunday</strong></p>  
<p><em>Do not be late</em></p>
```


Strong importance. A screen reader can change tone.

Stress emphasis — the word that changes the sentence's meaning.

 <i>

Bold and italic with no added meaning. A product name, a taxonomic term. **Not deprecated** — the living standard defines both; they just say nothing.

CYCLE 2 · THE VALIDATOR

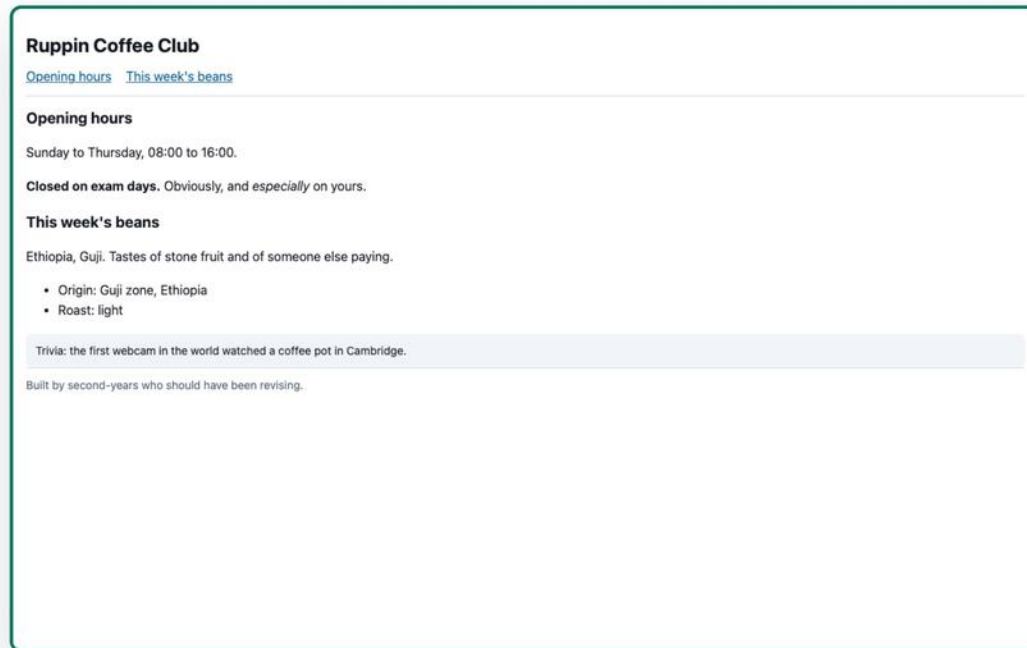
READING THE OUTPUT

1	72:8	error	Unclosed element '<section>'	close-order
2	79:8	error	Stray end tag '</div>'	close-order
3	81:20	error	Duplicate ID "hours"	no-dup-id
4	86:12	error	<p> element is not permitted as content under	element-permitted-content
5	91:6	error	End tag '</main>' seen but there were open elements	close-order
6	94:4	error	End tag '</body>' seen but there were open elements	close-order
7	95:2	error	End tag '</html>' seen but there were open elements	close-order

Three mistakes, seven lines. Fix the **first** one — the stray `</div>` — and five of these disappear on their own.

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS — EXACTLY AS IT DID BEFORE



Compare — `live-code/02-div-soup-to-landmarks/checkpoint.png` is this picture at 1280; `live-code/02-div-soup-to-landmarks/end/` is the code behind it. The pixels did not move. In the console, `$$('header, nav, main, footer').length` is 4, and the validator says nothing.

CYCLE 3 · 30 MIN

THREE BROKEN PATHS

Three images, three paths that worked on one computer in the world, three red rows in Network — and the validator says nothing. Then alt as a decision, the image that overflows a phone, a second page that links back, and the one safe way to open a new tab.

1 I DO

10 MIN

2 on the next two slides, 8 in the editor · live-code/03-three-broken-paths · file:/// · C:\, /images/ · alt="image" · width and height that fit 375 · about.html · rel="noopener noreferrer"

2 WE DO

8 MIN

you type it on your copy · compare with checkpoint 3 · Network with zero red rows

3 YOU DO

10 MIN

your own page: an image that loads, a second page that links back, one external link · checked in Network and at 375

4 DEBRIEF

2 MIN

the leading slash · who tells you the image loaded · ↗ reference

YOU DO · AI-FREE

CYCLE 3 · BEFORE YOU TYPE

"IT WORKS ON MY MACHINE"

The single most common way a week-1 submission arrives broken: images and links written as paths that exist on exactly one computer in the world.

WHAT I OPEN

Your page, on my laptop, from your repo. Not your Desktop. If the path only resolves on your machine, the mark is for a page with holes in it.

ONLY EVER WORKED ONCE

```
  
  

```

Three paths. Three red rows in the Network tab on any other computer — and the validator says nothing.

THREE KINDS OF PATH, AND ONLY ONE TRAVELS



`images/logo.svg`
relative to this file — travels with the folder ✓

`/images/logo.svg`
from the SERVER root — breaks in a subfolder ✗

`C:\Users\maya\Desktop\images\logo.svg`
one machine on earth has this folder ✗

`file:///Users/maya/Desktop/images/logo.svg`
not an address on the web at all ✗

All four are valid HTML. Only the first one is a working page.

IMAGES WITHOUT ALT ARE IMAGES THAT VANISH

NO ALT

```
1 
2
3 
```

DESCRIBED, OR EXPLICITLY SILENT

```
1 
4
5 
```

`alt=""` is a decision: "this picture carries no information, skip it". A missing `alt` is not a decision — the screen reader reads the file name.

LINKS SAY WHERE THEY GO

MEANINGLESS, AND UNSAFE

```
1 <a href="/pricing">Click here</a>
2
3 <a href="https://mdn.io" target="_blank">MDN</a>
```

DESCRIPTIVE, AND SAFE

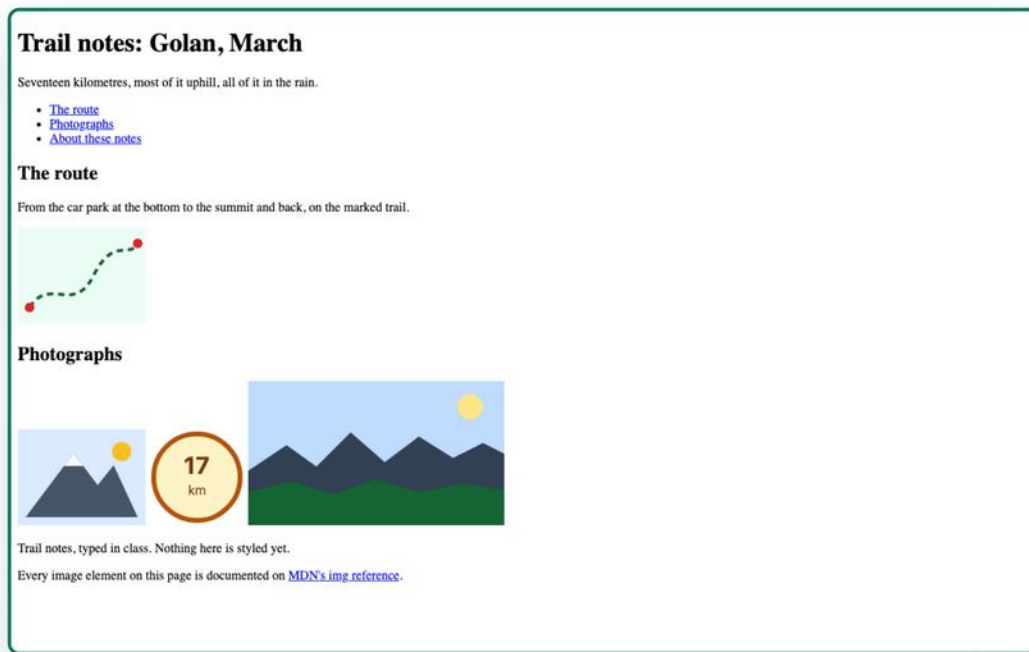
```
1 <a href="/pricing">View pricing</a>
2
3 <a href="https://mdn.io"
4   target="_blank"
5   rel="noopener noreferrer">MDN</a>
```

`target="_blank"` hands the new tab a reference back to your page.

`rel="noopener noreferrer"` takes it away.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/03-three-broken-paths/checkpoint.png` is this picture at 1280; `live-code/03-three-broken-paths/end/` is the code behind it. Four images, all loaded; Network with no red row; no horizontal scrollbar at 375; About opens and comes back.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, and MDN in the next tab.

CYCLE 4 · 25 MIN

ONE HONEST QUESTION: "SHOULD THIS SECTION BE AN ARTICLE?"

The ladder, the exam, and then one real question to Claude about the markup you typed. What comes back is mostly right, clearly written, and confidently wrong in one place — and MDN is one tab away.

1 I DO

7 MIN

2 on the ladder slide, 5 in the chat · ai-lab/04-honesty · P1 live, read aloud with MDN open · transcript.md if the room got something else

2 WE DO

6 MIN

everyone pastes P1 · find the claim that has no source before it is named

3 YOU DO

10 MIN

write P2 yourself · one question about your own markup · verify one claim · the first line of your journal

4 DEBRIEF

2 MIN

confidence is not a source · ↗ the journal exemplar

YOU DO · AI AS TUTOR

CYCLE 4 · THE LADDER, AND THE EXAM

THE LADDER IS A RULE ABOUT HOW YOU LEARN

The ladder is a rule about how you learn, not a rule we can police in a chat window. We will not read your chats. What we will do is sit you down in week 13, offline, with no AI, in front of the code you submitted, and ask you to fix a bug in it, add a feature to it and restructure a piece of it — for 20 % of your grade. If you can explain every line, the ladder did its job. If you cannot, no policy would have saved you. Every week from today has one exercise you do alone, so that week 13 is a Tuesday, not a shock.

Exam: 90 min · two versions, 2–3 days apart · tasks 32 / 33 / 31 + closing question 4 · your own SHA · offline · no AI. **Ladder:** tutor 1–5 · assistant 6–7 · partner 8–12. **Your first journal line** (`JOURNAL.md` , one line per AI cycle): *claim · what I checked · verdict.*

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you open the MDN page for `b` (or for whatever element the answer made a claim about) and see **for yourself** whether there is a Deprecated badge — instead of trusting how confident the answer sounded?
2. Does your P2 contain a constraint a browser or MDN can check — "name the MDN page for every claim", "do not say deprecated unless the page says so" — and does the answer meet it?
3. Does your journal line contain **a source or a tool** — an MDN page title, validator output, what you typed in the console — and not just "Claude was right / wrong"?

Compare — `ai-lab/04-honesty/checkpoint-he.md` has the same three questions; `artifact/claude-p1/` and `artifact/claude-p2/` are the two answers, ready to read side by side.

CYCLE 5 · 20 MIN

FROM INIT TO SHA

A private repo from the template, me in Collaborators, a clone, three commits each with a change you can see, a push, and the forty characters that freeze a submission. Plus the file you almost committed forever.

1 I DO

7 MIN

2 on the next slide, 5 in the terminal · live-code/05-init-to-sha · clone · status, add, commit · git add · without looking, and the fix · three commits · push · git log -1 --format=%H

2 WE DO

5 MIN

your repo, your clone, your page in week-01/, one real commit, push

3 YOU DO

6 MIN

two more commits you can see · the SHA and the URL in a scratch file — the dry run · AI as tutor on what the commands mean

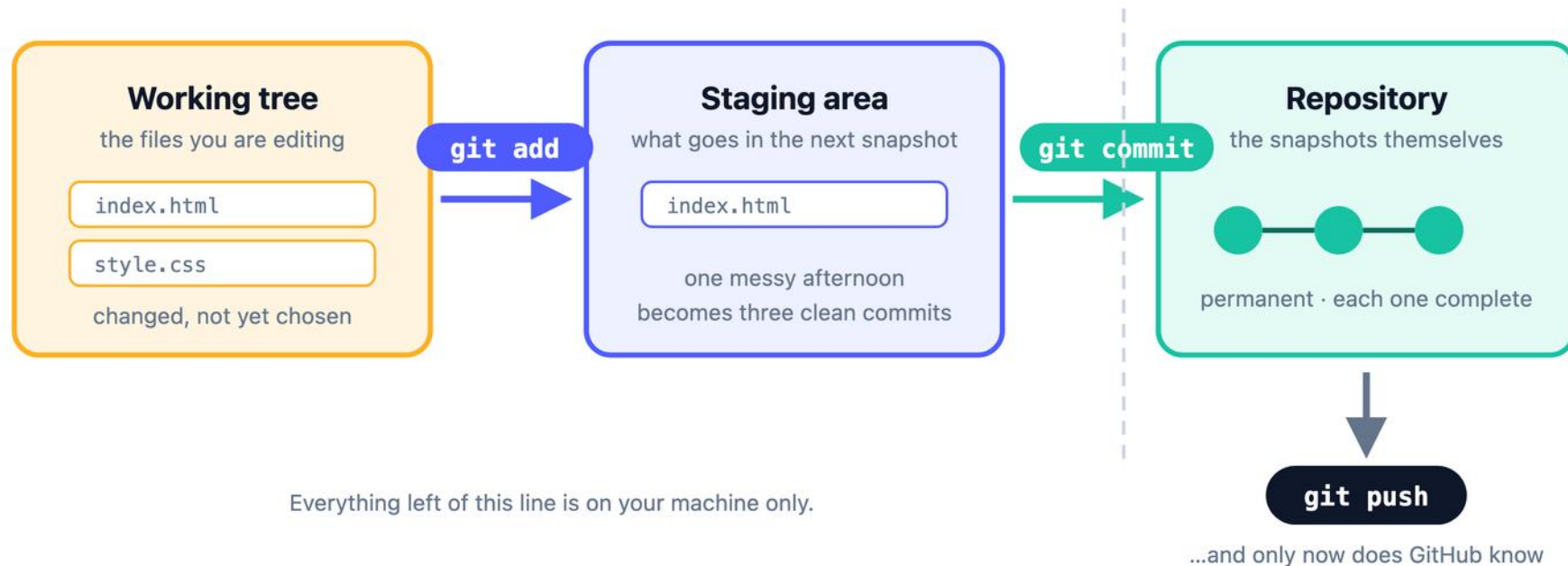
4 DEBRIEF

2 MIN

what you paste into Moodle, and why two things · ↗ reference

YOU DO · AI AS TUTOR

THREE PLACES A FILE CAN BE



A COMMIT MESSAGE IS FOR THE PERSON READING IT IN WEEK 13

NO INFORMATION

```
update  
fix  
asdf  
final version
```

A READABLE HISTORY

```
feat: add contact form with labelled fields  
fix: correct relative path to logo  
docs: add README with run instructions  
style: format with Prettier  
refactor: extract nav into its own section
```

Small commits, often. One giant commit at the end is not a history — it is a backup.

YOUR REPO, AND WHY I AM IN IT

```
1 # one PRIVATE repo for the whole semester, from the course template
2 frontend-2027-<student-id>
3
4 git clone git@github.com:<you>/frontend-2027-<student-id>.git
5
6 git add . && git commit -m "feat: week 1 personal page"
7
8 git push
```

Private

Forty public repos in one course is a copy-paste machine.

Collaborators

Settings > Collaborators > Add people — add me in week 1, not at the end. Pending invite is full marks: the invitee accepts, and that is me, tonight. In week 13 I pre-load your own project onto the exam machines — and on exam day there is no fixing this.

THE SHA FREEZES THE SUBMISSION

```
1 git log -1 --format=%H
2 9f4c1a7e0b2d5836a1c4f7e9d2b8a6c3e5f10947
3
4 # submit BOTH, in Moodle:
5 # https://github.com/<you>/frontend-2027-<student-id> + the SHA
```

What is in that commit is what gets marked. Pushing after the deadline does not change the grade, and there is no argument about it.

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS — AND YOUR TERMINAL SHOULD BE CLEAN

Noa Levi

Third-year information systems student. Fluent in C# and SQL, new to the browser.

- [Right now](#)
- [The plan](#)

Right now

This page was typed from an empty file, one line at a time, in the first hour of the course.

The plan

- What I already write: C#, SQL, a little Python.
- What I am learning now: what the browser does with the text I send it.
- What I want by week 12: one application I can explain line by line.

Frontend Development 2027, Ruppin Academic Center. Typed in class.

Every commit behind this page is on [Github](#), in a private repository.

Compare — `live-code/05-init-to-sha/checkpoint.png` is the page after the third commit; `live-code/05-init-to-sha/end/` is the code behind it. `git status clean`, `git log --oneline` with your commits above the template's, and your commit visible on the repository page.

ASSIGNMENT · 145-175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Your private repo from the template, me in Collaborators, and `week-01/index.html` on your own content: the skeleton (cycle 1), header · nav list · main · sections · footer (cycle 2), one image that loads, with a real alt (cycle 3). Validate clean. Three commits.

EXTEND · STRETCH AND CHALLENGE

A second page that links back (cycle 3), an `article` or `aside` (cycle 2), no horizontal scroll at 375, `strong` / `em` and no `b` / `i`, one safe external link. You typed every one of them today.

THE RULES, AND THE EXAM

No CSS. No JavaScript. No paths that only work on your machine.

AI as tutor — it explains, you type. Submit the repo URL and the SHA in Moodle. Week 13 asks you about this code, offline.

THE WHOLE VOCABULARY, ON ONE PAGE

DOCUMENT

html · head · body · title · meta ·
link · script · style

LISTS

ul · ol · li · dl · dt · dd

FORMS

form · fieldset · legend · label ·
input · textarea · select ·
option · button

SECTIONING

header · nav · main · section ·
article · aside · footer · h1–h6

LINKS AND MEDIA

a · img · picture · source ·
figure · figcaption · video ·
audio · svg · iframe

INTERACTIVE

details · summary · dialog ·
progress · meter

TEXT

p · strong · em · b · i · small ·
mark · time · code · pre ·
blockquote · abbr

TABLES

table · caption · thead · tbody ·
tfoot · tr · th · td

MEANINGLESS ON PURPOSE

div · span — and that is the entire point of
them.

About 110 elements exist. Today you typed the first three columns; forms and tables are next week. **This page is in the reference deck too — keep it open.**

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

1. The browser received your HTML. Say the six words, in order — and which one the validator lives in.
2. Your page and your image are in the same folder. Write the `src`. Now write the one that only works on your machine — and say what the Network tab shows for it.
3. You have committed and pushed. What exactly do you paste into Moodle, and why two things?

► **GO DEEPER** — the reference deck is optional depth, and it is not examined: how the parser builds the tree, every kind of href, the void elements, the characters you have to escape, the forms and tables you meet next week, and what git actually stores. The next slide lists it.

GO DEEPER

The reference deck for this week — every slide the lecture pointed at, in order.

[1 · The HTML catalogue](#) ← spine 36

[11 · How the parser builds the tree](#) ← spine 6

[14 · The skeleton, line by line](#) ← spine 6

[19 · Void elements — no content, no closing tag](#) ← spine 6

[25 · The semantic elements, all of them](#) ← spine 10

[29 ·
 is not spacing](#) ← spine 10

[32 · Lists — three kinds, and they nest](#) ← spine 13

[34 · What the validator's close-order cascade means](#) ← spine 10

[35 · Anchors — every kind of link](#) ← spine 13

[37 · The href catalogue](#) ← spine 18

[39 · A page is always opened with Live Server](#) ← spine 18

[40 · figure, figcaption — and time, next week](#) ← spine 18

[50 · What git actually stores: blob, tree, commit](#) ← spine 28

[53 · .gitignore patterns — the ones in your template](#) ← spine 28

[54 · Commit message prefixes — the convention this course reads](#) ← spine

28

[55 · Where to look things up](#) ← spine 25